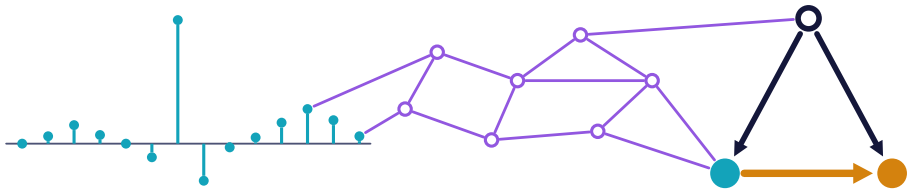
Experimentos biomédicos reproducibles

Del dato original a la evidencia

P. E. Caicedo-Rodríguez
Centro de Estudios en Biomédica y Biotecnología
17 de septiembre de 2026



El problema



Un resultado que nadie puede rehacer

- El archivo crudo se sobrescribió al «limpiarlo»
- Un paso se hizo a mano en una hoja de cálculo
- Los faltantes se llenaron con la media, sin dejar rastro
- Existe `resultados_final_v3.xlsx`, pero no el código que lo produjo

Reproducible no es lo mismo que correcto

- **Reproducible**: los mismos datos y el mismo código dan el mismo resultado
- Es necesario para revisar un análisis, pero no lo hace correcto
- Sin reproducibilidad, ni siquiera se puede saber si es correcto

Cuatro reglas



Lo que se decide antes de analizar

1. **El dato original no se toca**: solo lectura, con su huella SHA-256
2. **Cada transformación queda escrita**: código y contrato, no clics
3. **Nada se inventa**: los faltantes se conservan con su motivo
4. **La unidad de análisis se fija antes** de mirar los resultados

> ⚠ **Identificadores personales**
>
> Los archivos con datos personales no entran a `data/raw`: primero se seudonimizan.

BioForge



Cuatro capas, una sola dirección

Contrato versionado (YAML / JSON Schema, Pydantic): columnas, unidades y rangos



Manifiesto: SHA-256 de cada archivo y versiones del contrato y del código

El dato fluye en un solo sentido; cada capa se puede regenerar desde la anterior.

Un contrato dice qué es válido

```yaml
# Ilustrativo: contrato de una tabla de parámetros de marcha
contrato: parametros-de-marcha
version: 0.1.0
unidad_de_analisis: participante
columnas:
  - {nombre: velocidad_marcha, unidad: m/s, rango: [0.1, 2.5]}
  - {nombre: cadencia, unidad: pasos/min, rango: [40, 180]}
faltantes: conservar          # no se imputan
```

- El contrato se versiona como el código: cambiarlo es una decisión registrada

Un manifiesto dice qué se usó

```python
import hashlib

def sha256(ruta):
    h = hashlib.sha256()
    with open(ruta, "rb") as f:
        while trozo := f.read(1 << 20):
            h.update(trozo)
    return h.hexdigest()
```

- Una huella por archivo, más la versión del contrato y del código

Del manifiesto al informe

```
uv sync                       # el mismo entorno
uv run python figuras.py      # figuras desde derived/
uv run quarto render          # el informe
```

Para llevarse



Tres hábitos

1. Guardar el original y su huella antes de hacer cualquier otra cosa
2. Escribir cada transformación como código que se pueda volver a correr
3. Decir lo que el análisis mide y lo que no, también en el informe